SOUTH LAKE HOME EDUCATORS



Lesson 12

Hearing It Happen

Semester 1, Week 12 - Sound Effects

At a Glance

Lesson 12 objectives & materials

By the end of today, students will:

- Wire sound cues into their exact matching code paths
- Recognize why event-driven triggers need no replay guard
- Formalize six lessons' worth of optional sound ideas
- Play a full round and hear every outcome distinctly

What We're Using Today

- Demo laptop with all 8 engines, audio output working
- GameMaker (GML Visual), for the live lecture demo
- One new sound asset: sfx_wave.wav
- Everything else reuses Lessons 2-8's optional sound ideas
- Your own [YourName]Game project from Lesson 11 - not a new one

IGDA framework tie-in: Audio - sound design fundamentals (introductory); Programming - event-driven triggers, code reuse (intermediate).

Today's Agenda



0:00 - 0:05	Welcome back, Lesson 11 recap, the polish phase begins
0:05 - 0:15	The Sound Payoff: One New Sound, Several Existing Moments
0:15 - 0:35	A Dedicated Sound for Every Outcome
0:35 - 0:45	Why None of Today's Sounds Need a Retrigger Guard
0:45 - 0:55	Introduce lab exercise, hand out engine-specific handouts
LAB TIME	***— Hearing It Happen —***
0:55 - 1:50	Wire up every sound, test a full round by ear
1:50 - 2:00	Exit check: play a round for the class



Welcome Back

Level 2 Complete is done. The polish phase begins.

5 min

Welcome Back

5 min

Quick recap of Lesson 11:

- Level 2 Complete reached - a real, replayable game with multiple waves
- No new mechanics from here through Lesson 15 - just making the loop feel good
- Today: sound. Next: animation, difficulty scaling, and a final polish pass



Open your own project from last week today, not a new one. One new sound file is provided - sfx_wave.wav - everything else may already be in your project from earlier optional ideas.



The Sound Payoff

One new sound, several moments you already built.

10 min

One New Sound, Several Existing Moments

10 min



Since Lesson 2, six lessons quietly offered an optional 'add a sound' idea. Today, every one of those sounds becomes required for everyone - shoot, explosion, life lost, and game over - plus one brand-new sound, matching the capstone's Level 3 feature list exactly.

No new logic today - just one more line at each place you already built.

- Shoot - when you fire
- Explosion - when a hit lands
- Wave Clear - a non-final wave ends (NEW)
- Life Lost / Game Over - a life or the round ends



A Dedicated Sound for Every Outcome

One outcome, one sound, wired directly into its branch.

20 min

One Outcome, One Dedicated Sound

20 min

Shoot and hit sounds are easy - one sound each, already wired since Lessons 3-4. For win, wave-clear, life-lost, and game-over: each gets its own dedicated sound, wired directly into the branch that already decides it.

GameMaker demo: on obj_bullet's win-check (from Lesson 11), add a Play Audio block for sfx_win on the True output and sfx_wave on the False output. On obj_enemy's escape branch, confirm sfx_gameover/sfx_lifelost (from Lesson 8) are already wired in.

Every engine in this course uses the same dedicated-per-event pattern - no shared sound player swapping between clips anywhere. Simple, consistent, easy to audit later.



No Retrigger Guard Needed

Why today's sounds are safe by construction.

10 min

Why No Retrigger Guard?

10 min



Lesson 2's beep-beep needed an explicit stop call because it was continuous. Today's sounds are one-shot - they finish on their own. But there's a second reason they're safe: every one lives inside a branch that only runs once per actual event.



TIC-80 note: Lesson 6's optional win sound may have needed a hasWon guard flag back then, since the check ran every frame. Lesson 10/11's restructuring moved it into a one-time hit branch - that old guard flag can be removed now if it's still there.

Good structure, built for one reason, quietly solves a second problem later.



Today's Exercise & Lab

Hearing It Happen - wiring up every sound.

10 min

Hearing It Happen

10 min

Wire every sound into the exact branches that already represent their events - built on top of your existing project.

Level 1 - Minimal

Every expected sound plays at the correct moment, at least once each.

MVP - a complete finish for today.

Level 2 - Complete

Every sound fires reliably every time, none interfere with each other.

Everything works end to end.

Level 3 - Refined

Add the optional win sound, pitch/volume variety, or tidy the code.

Ready to share.



Play a full round OUT LOUD today - a shot, a hit, an early wave clear, and separately a lost life and a loss - and confirm every sound genuinely different from the others.

Lab Time Notes



- Remind students who skipped the optional Lessons 2/3/4/6/8 ideas that today's Handout gives them full instructions, not just "add one sound."
- Watch for a sound call placed outside the correct if/else branch - causes two round-ending sounds to play together on the same frame.
- Listen as you circulate this week, not just watch the screen - a wrong or missing sound is often faster to catch by ear than by reading code.
- This is a lighter week - if a cluster finishes early, point them at Level 3 ideas rather than moving ahead to next week's material.

Protect Your Project

A few students have already lost work this way - worth two minutes before we wrap up today.

Game engines and IDEs are constantly writing and maintaining files behind the scenes - not just when you hit Save. If your computer sleeps, hibernates, or loses power WHILE one of those files is being written, that file can end up corrupted - and a corrupted project file often means the whole game won't load again.

This isn't rare or exotic - it's the single most common way students have lost work in this course so far.

The good news: the fix is a habit, not a tool. The next slide covers the four steps that prevent this entirely.

Your Backup Habit

4 steps

1. Battery backup	Desktop: a UPS, even a small one, just enough to shut down cleanly. Laptop: a laptop with an actual working battery - don't run a laptop with a dead/removed battery on wall power alone.
2. Save often	File > Save, Ctrl-S, or your engine's own save command - don't wait until you're "done" for the day.
3. Clean exit, every time	Fully close the engine/IDE before powering off, sleeping, or walking away - never just close the lid or pull the plug while it's still open.
4. Back up after shutdown	Once the application is fully closed, back up your project - every session, not just occasionally.

Backup methods: your engine's own export/import, a plain copy of the project folder, or a version control system like git (local and/or GitHub/GitLab). Full engine-by-engine instructions and a git mini-tutorial are in the Backups Sidebar handout.



It Sounds Like a Game!

A shot, an explosion, a wave clear, a life lost, a game over - five moments, five distinct cues.

Next week: basic animation - an explosion effect, a hit flash/reaction.

Keep playtesting a full round at home if you'd like - try a Level 3 sound idea if you haven't already.